Cisco ISE Device Compliance & Posture Integration with Intune (MDM Compliance Connector)

Implementation Solution Guide

Version 1.0.0 · 2026-07-01

Contents

Executive Summary 4

The problem . . . 4

The solution . . . 4

Scope and vendor boundary . . . 4

How it's deployed . . . 4

What you get . . . 4

The bottom line . . . 4

Solution Overview 5

What this solution does . . . 5

The moving parts . . . 5

How the signal flows . . . 5

The compliance and enrollment signals ISE reads . . . 6

How signals map to authorization . . . 6

Named objects this solution creates . . . 6

Deployment model . . . 7

Requirements 7

R1. Microsoft Intune with enrolled devices . . . 7

R2. Intune device compliance policies in place . . . 8

R3. An Entra app registration with the right Graph permissions . . . 8

R4. Cisco ISE licensed and versioned for MDM integration . . . 8

R5. Network reachability and trust: ISE → Microsoft Graph . . . 9

R6. A pilot group . . . 9

R7. Change control and a documented rollback . . . 9

Implementation 10

Step 1 — Register the app in Entra that ISE will use . . . 10

Step 2 — Grant the app the Microsoft Graph permissions ISE needs . . . 10

Step 3 — Create a client secret for the app . . . 11

Step 4 — Connect ISE to Intune (architectural — defer to Cisco docs) . . . 11

Step 5 — Define the named compliance policies in Intune . . . 11

Step 6 — Map compliance to ISE authorization (architectural — defer to Cisco docs) . . . 12

Step 7 — Assign to the pilot and stage the gate . . . 12

Step 8 — Validate (pass/fail gate) . . . 13

Step 9 — Expand . . . 13

Testing 13

Gate 1 — Connector readiness . . . 13

Gate 2 — Compliance policies present and evaluating . . . 14
Gate 3 — Compliant device is GRANTED access (the positive path) . . . 14
Gate 4 — Non-compliant device is DENIED access (the critical path) . . . 15
Gate 5 — Unmanaged device is DENIED / redirected . . . 15
Gate 6 — Fleet coverage snapshot . . . 15
Exit criteria to expand . . . 15
Troubleshooting 16
1. App registration / permission errors . . . 16
2. Connector not reporting (ISE reads no compliance state) . . . 16
3. Compliant device denied (false lockout — positive path broken) . . . 17
4. Non-compliant device allowed (enforcement gap — critical) . . . 18
5. Quick reference — which tool for which symptom . . . 18
Best Practices 18
Grace periods — enforce, but don't ambush . . . 19
Remediation — give users a path back . . . 19
Naming and reuse . . . 19
Secret hygiene . . . 19
Pilot-first, always . . . 19
Steady-state operations . . . 20
Deliverables Summary 20
Documentation . . . 20
PowerShell scripts . . . 21
Named objects this solution standardizes . . . 21
Outcome . . . 22
Appendix A — PowerShell Scripts 23

Executive Summary

The problem

A device can present a valid network certificate and still be unsafe — unencrypted, unpatched, jailbroken, or no longer managed. A certificate proves *who* a device is; it says nothing about whether the device is *healthy*. Networks that authorize on certificates alone are letting compromised-but-credentialed devices onto VPN and internal segments.

The solution

This solution wires **Cisco ISE** to **Microsoft Intune** through the **MDM compliance connection** so that ISE asks Intune, in real time, whether each connecting device is **enrolled and compliant**, and gates network / VPN access on the answer. Compliance policy becomes enforced access: compliant devices are granted, non-compliant and unmanaged devices are denied or quarantined.

ISE authenticates to your Microsoft Entra tenant using a dedicated app registration and reads device state over Microsoft Graph. This is the posture / NAC layer that turns Intune's compliance verdict into a network access decision.

Scope and vendor boundary

The **Intune, Entra, and Microsoft Graph** side is delivered at exact, click-by-click detail: the app registration ISE uses, the Graph permissions, the named compliance policies, the pilot scoping, and the pass/fail validation. The **Cisco ISE console** internals (External MDM server setup and authorization policy editing) are described at the architectural level and **deferred to Cisco's official documentation** — ISE menus change by release and are followed from the vendor, not fabricated here.

How it's deployed

The standard staged model with explicit pass/fail gates: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.** The gate that matters most is proven on the pilot before any expansion: **compliant devices are granted access; non-compliant devices are denied.** The change is rollback-ready throughout.

What you get

Full documentation (Overview, Requirements, Implementation, Testing, Troubleshooting, Best Practices) plus **five PowerShell scripts** — connector-readiness discovery, per-device compliance/enrollment validation, a fleet coverage and access-decision report, safe remediation (default `-WhatIf`), and a pilot-group builder. All scripts are read-only against Microsoft Graph.

The bottom line

Network and VPN access granted only to devices Intune says are compliant and enrolled — enforced by Cisco ISE, and proven on a pilot before you scale. Self-guided, tested, at a fraction of consulting cost.

Solution Overview

What this solution does

A device can hold a valid network certificate and still be dangerous: unencrypted, out of date, jailbroken, or unenrolled from management. A certificate proves *identity*; it says nothing about *health*. This solution closes that gap by wiring **Cisco ISE** (Identity Services Engine) to **Microsoft Intune** so that ISE asks Intune, in real time, *"Is this exact device enrolled and compliant right now?"* and grants or denies network / VPN access on the answer.

The link between them is the **MDM compliance connection** — ISE's built-in ability to call a Mobile Device Management provider (here, Intune) over its Graph API. ISE authenticates to your Microsoft Entra tenant using a dedicated **Entra app registration**, reads each device's enrollment and compliance state, and feeds that state into its authorization rules as just another condition. Compliant and enrolled devices pass; non-compliant or unmanaged devices are denied or quarantined.

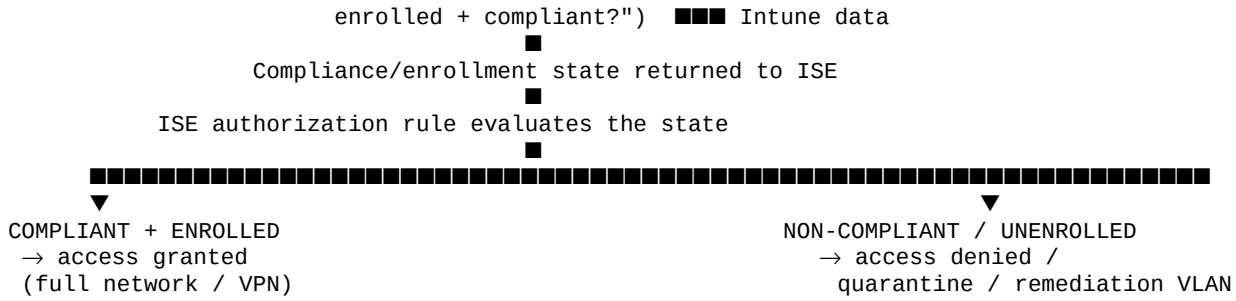
This document explains how the pieces fit. The exact click-by-click work on the **Intune, Entra, and Graph** side is authored in full in this solution. The **Cisco ISE console** side (External MDM server setup, authorization policy editing) is described at the architectural level and **deferred to Cisco's official documentation** — ISE menu paths change between releases and must be followed from the vendor's current docs, not fabricated here.

The moving parts

Component	Role in this integration
Microsoft Intune	Source of truth for device enrollment and compliance. Compliance policies define what "healthy" means.
Microsoft Entra ID	Hosts the **app registration** ISE uses to authenticate, and holds the Graph API permissions that authorize ISE to read device state.
Microsoft Graph	The API surface ISE calls to read `managedDevices` / compliance state. Intune data is exposed through Graph.
Cisco ISE	The policy enforcement brain. Holds the **External MDM server** definition pointing at Intune, and the **authorization rules** that turn compliance into an access decision.
Network access devices (VPN headend, switches, wireless controllers)	Enforce the decision ISE returns — grant, deny, or quarantine.

How the signal flows

```
Device connects (VPN / 802.1X / wireless)
        ■
        ▼
Network access device  ■■RADIUS■■■  Cisco ISE
                                        ■
                    ISE authenticates the app registration to Entra
                                        ■
                          ISE calls Microsoft Graph ("is this device
```



The compliance and enrollment signals ISE reads

ISE does not re-evaluate the device itself. It reads the verdict Intune has already computed and the enrollment facts Intune holds. The signals that matter for an access decision are:

- **Registration / enrollment state** — is the device enrolled in Intune (MDM-managed) at

all? An unenrolled device is unmanaged and should not be trusted.

- **Compliance state** — Intune's rolled-up `compliant` / `noncompliant` / `inGracePeriod`

verdict for the device, computed from every assigned compliance policy.

- **Per-signal posture** — the underlying facts that drive compliance, such as disk

encryption on, OS/patch level at or above a floor, no jailbreak / root, and a required password / PIN. These are configured as **compliance policy** rules in Intune (authored in this solution) and roll up into the single compliance verdict ISE consumes.

Because Intune computes compliance continuously, the decision ISE enforces is **real-time**: a device that drifts out of compliance (encryption disabled, patch missed) flips to non-compliant in Intune, and its *next* ISE-mediated access attempt is denied.

How signals map to authorization

On the ISE side, the MDM compliance dictionary attributes (device registered, device compliant, and related fields) become **conditions** inside ISE authorization rules. The mapping pattern this solution standardizes on is:

- **Registered = true AND Compliant = true** → authorization result **grants** full access.
- **Registered = true AND Compliant = false** → authorization result **denies** or sends the

device to a **remediation / quarantine** VLAN with limited reachability.

- **Registered = false (unmanaged)** → authorization result **denies** or redirects the user

to an enrollment portal.

The specific ISE authorization-profile and rule construction is done in the ISE console per Cisco's documentation. This solution provides the exact Intune/Entra/Graph configuration that makes those signals *available and trustworthy*, the named compliance policies that define "compliant," the pilot scoping, and the pass/fail validation that proves non-compliant devices are actually denied before you expand.

Named objects this solution creates

To keep the build repeatable and auditable, every object is named to a fixed convention and reused verbatim throughout the guides:

- **Entra app registration:** `ISE-Intune-Compliance-Connector`
- **Compliance policies:**
- `Compliance – Windows – Baseline`
- `Compliance – iOS – Baseline`
- `Compliance – Android – Baseline`
- `Compliance – macOS – Baseline`
- **Pilot group:** `PILOT – ISE Posture Devices`
- **Expansion group:** `PROD – ISE Posture Devices`

Deployment model

This solution follows the standard staged model with explicit pass/fail gates:

Assess → **Configure** → **Pilot** → **Validate** → **Expand.**

- **Assess** — confirm prerequisites (compliance policies, licensing, reachability).
- **Configure** — register the app, grant Graph permissions, connect ISE to Intune, author

the named compliance policies, and build the ISE authorization mapping.

- **Pilot** — assign to `PILOT – ISE Posture Devices` only.
- **Validate** — prove compliant devices are granted access and non-compliant devices are

denied (the pass/fail gate).

- **Expand** — roll out to `PROD – ISE Posture Devices` and the fleet.

Requirements

Confirm each prerequisite below **before** you begin the implementation. These are specific to the ISE ↔ Intune posture integration and are **in addition to** the category-level Network global requirements (see `global-requirements-network.md`, especially an Entra tenant, Intune, admin roles, a pilot group, and change control).

Each requirement is written as **Description / Why / Verify / Implement if missing.** Work through them top to bottom; do not start Configure until all are green.

R1. Microsoft Intune with enrolled devices

Description: An active Intune tenant with the target devices already enrolled (MDM-managed).

Why: ISE reads *Intune's* view of each device. A device Intune does not manage has no enrollment or compliance state for ISE to query, so it can only ever be treated as unmanaged.

Verify: Sign in to `https://intune.microsoft.com` → **Devices** → **All devices** and confirm your target devices appear with a **Managed by = Intune (MDM)** and a recent check-in time.

Implement if missing: Assign an Intune-bearing license to the users, then enroll the devices (Windows: Entra join + Intune enrollment; iOS/Android: Company Portal; macOS: ADE or Company Portal). Do not proceed until at least the pilot devices are enrolled.

R2. Intune device compliance policies in place

Description: At least one **compliance policy** per target platform, defining what "compliant" means (encryption, OS floor, jailbreak/root block, password/PIN).

Why: ISE's access decision is only as meaningful as the compliance rules behind it. With no compliance policy, every enrolled device is trivially "compliant" and the posture gate enforces nothing. This solution creates the named baseline policies in the Implementation guide.

Verify: Intune → **Devices** → **Compliance** → **Policies**. Confirm a policy exists for each platform you will gate. If you are starting fresh, this will be empty — that is expected; you will create them.

Implement if missing: Create the named baseline policies during Implementation: `Compliance – Windows – Baseline`, `Compliance – iOS – Baseline`, `Compliance – Android – Baseline`, `Compliance – macOS – Baseline`.

R3. An Entra app registration with the right Graph permissions

Description: A dedicated Microsoft Entra **app registration**, `ISE-Intune-Compliance-Connector`, with a client secret and **application** Microsoft Graph permissions that let ISE read managed device / compliance state — at minimum `DeviceManagementManagedDevices.Read.All` and `DeviceManagementConfiguration.Read.All` — admin-consented.

Why: ISE authenticates to your tenant *as this app* and calls Graph with its permissions. Without the app registration and consented permissions, ISE cannot read device state and the connector will fail to report.

Verify: Entra → **App registrations** → look for `ISE-Intune-Compliance-Connector`. On **API permissions**, confirm the Graph permissions are present and show **"Granted for <tenant>"** (green check). Confirm a valid (unexpired) client secret exists.

Implement if missing: Create it during Implementation (register the app, add the Graph application permissions, grant admin consent, generate a client secret). The `ISEPOST-Discovery.ps1` script reports on this app's readiness.

R4. Cisco ISE licensed and versioned for MDM integration

Description: A running Cisco ISE deployment on a release that supports the **MDM / External MDM (Intune)** integration, with the appropriate licensing (the tier that includes MDM/posture).

Why: The External MDM server feature and its authorization dictionary attributes are only present on supported ISE releases and license tiers. This is the enforcement engine — without it there is nothing to consume the compliance signal.

Verify: In the ISE admin console, confirm the running version and installed licenses. **Follow Cisco's official documentation** for the exact menu path and the minimum version / license that supports Intune MDM integration for your release — do not assume from this guide.

Implement if missing: Upgrade ISE and/or apply the required license per Cisco's documentation and your Cisco account team. This is a Cisco-side action.

R5. Network reachability and trust: ISE → Microsoft Graph

Description: ISE must be able to reach Microsoft Graph / Entra login endpoints over HTTPS (443), with correct DNS resolution, egress, and a trusted certificate chain to Microsoft.

Why: The connector is an outbound API call from ISE to Microsoft's cloud. Blocked egress, a proxy that intercepts TLS, or a missing root CA in ISE's trust store breaks the connection and ISE cannot read compliance state.

Verify: From the ISE deployment's egress path, confirm HTTPS reachability to Microsoft Graph and Entra login endpoints and that the Microsoft server certificate chain is trusted by ISE. (In the ISE console, the External MDM server **test connection** is the authoritative check — see Cisco's documentation.)

Implement if missing: Open outbound 443 to the Microsoft Graph / Entra endpoints, fix DNS, and import the required Microsoft root/intermediate CA certificates into the ISE trusted certificate store per Cisco's documentation.

R6. A pilot group

Description: A Microsoft Entra security group, `PILOT – ISE Posture Devices`, containing only the initial pilot devices (or their users).

Why: Compliance policies and the posture gate are rolled out pilot-first so a mistake denies access to a handful of test devices, not the fleet. This is the safety valve for a live-access change.

Verify: Entra → **Groups** → confirm `PILOT – ISE Posture Devices` exists and contains the intended pilot members.

Implement if missing: Create it in Entra (**Groups** → **New group** → **Security**) or run the `ISEPOST-Set-ScopeTargeting.ps1` script in this solution, which creates and populates it.

R7. Change control and a documented rollback

Description: An approved change window and a written rollback for the posture gate (loosen the ISE authorization rule to permit, or unassign the compliance policy) before you enable enforcement.

Why: This change gates live network and VPN access. A misconfigured rule can lock out legitimate users. You must be able to revert quickly.

Verify: A rollback procedure is written and approved: how to set the ISE authorization rule back to permit, and how to unassign / scope out the compliance policy, with who approves and how users are notified.

Implement if missing: Document the rollback (ISE rule revert + compliance policy unassign + comms plan) and get it approved before Pilot.

Implementation

Follow these steps in order. They implement the **Configure** and **Pilot** phases of the deployment model. The **Intune / Entra / Graph** steps are click-by-click; the **Cisco ISE** steps are described at the architectural level and **deferred to Cisco's official documentation** for the exact console paths.

Reuse the object names exactly as written — later steps and the PowerShell scripts depend on them:

- App registration: `ISE-Intune-Compliance-Connector`
- Compliance policies: `Compliance – Windows – Baseline`, `Compliance – iOS – Baseline`, `Compliance – Android – Baseline`, `Compliance – macOS – Baseline`
- Pilot group: `PILOT – ISE Posture Devices`
- Expansion group: `PROD – ISE Posture Devices`

Step 1 — Register the app in Entra that ISE will use

1. Sign in to `https://entra.microsoft.com` as an administrator (Application Administrator or Global Administrator).
1. Go to **Identity** → **Applications** → **App registrations** → **+ New registration**.
2. **Name:** `ISE-Intune-Compliance-Connector`.
3. **Supported account types:** *Accounts in this organizational directory only (single tenant).*
1. Leave **Redirect URI** empty. Click **Register**.
2. On the app's **Overview** page, copy and save the **Application (client) ID** and the **Directory (tenant) ID** — you will enter both into ISE later.

Step 2 — Grant the app the Microsoft Graph permissions ISE needs

1. In the app, go to **API permissions → + Add a permission → Microsoft Graph → Application permissions** (not Delegated — ISE calls Graph as itself, unattended).
1. Add these application permissions:
- `DeviceManagementManagedDevices.Read.All` — read managed device + compliance state.
- `DeviceManagementConfiguration.Read.All` — read compliance policy configuration.
1. Click **Add permissions**.
2. Click **Grant admin consent for <your tenant>** and confirm. Every permission must show **"Granted for <tenant>"** with a green check before you continue.

> *Note: Cisco's documentation lists the exact permission set for your ISE release. The two read permissions above cover reading device compliance/enrollment. If your ISE version's External MDM*

flow requires an additional permission, add it here the same way and re-consent.

Step 3 — Create a client secret for the app

1. In the app, go to **Certificates & secrets** → **Client secrets** → **+ New client secret**.
2. **Description:** `ISE connector secret`. **Expires:** choose a policy-approved lifetime

(for example 12 months) and record the renewal date.

1. Click **Add**, then **immediately copy the secret Value** (not the Secret ID). It is shown

only once. Store it securely — you will paste it into ISE.

Step 4 — Connect ISE to Intune (architectural — defer to Cisco docs)

In the **Cisco ISE** admin console, add Intune as an **External MDM server** using the credentials from Steps 1–3. At the architectural level you will supply ISE with:

- The **Directory (tenant) ID** (Step 1).
- The **Application (client) ID** (Step 1).
- The **client secret Value** (Step 3).
- The Microsoft Graph / Entra endpoints ISE authenticates and queries against.

Then use ISE's **Test Connection** on the External MDM server definition and confirm it succeeds — a successful test proves the app registration, permissions, secret, and network reachability (R5) are all correct.

> ***Vendor boundary:*** *the exact ISE menu path (for example the External MDM / MDM & UEM server area), field labels, and the connection-test button vary by ISE release.* ***Follow Cisco's official ISE documentation for your version*** *to enter these values and run the test. Do not proceed to Step 6 until Cisco's Test Connection reports success.*

Step 5 — Define the named compliance policies in Intune

Create one baseline policy per platform. These define what "compliant" means and are the substance behind ISE's access decision. For each policy: Intune → **Devices** → **Compliance** → **Policies** → **+ Create policy** → select the platform → name it exactly as below → set the rules → set the **Actions for noncompliance** → **Assign** to `PILOT – ISE Posture Devices` → **Create**.

`Compliance – Windows – Baseline`

- Platform: **Windows 10 and later**.
- Rules (Compliance settings): require **BitLocker**, require **Secure Boot**, set a

minimum OS version floor, require **Microsoft Defender Antimalware** on, require a **password/PIN** to unlock.

- Actions for noncompliance: **Mark device noncompliant** immediately (or after a short grace

— see Best Practices), and optionally **Send email to end user**.

`Compliance – iOS – Baseline`

- Platform: **iOS/iPadOS**.
- Rules: **Jailbroken devices = Block**, **minimum OS version** floor, require a **passcode**, require **device encryption** (implicit with passcode on iOS).
- Actions for noncompliance: **Mark device noncompliant** (with grace per policy).

`Compliance – Android – Baseline`

- Platform: **Android Enterprise** (work profile / fully managed as applicable).
- Rules: **Rooted devices = Block**, require **Play Integrity / SafetyNet** device health, **minimum OS version** floor, require **encryption of data storage**, require a **password/PIN**.
- Actions for noncompliance: **Mark device noncompliant** (with grace per policy).

`Compliance – macOS – Baseline`

- Platform: **macOS**.
- Rules: require **FileVault** disk encryption, set a **minimum OS version** floor, require a **password** to unlock.
- Actions for noncompliance: **Mark device noncompliant** (with grace per policy).

> *Assign every baseline policy to* `PILOT – ISE Posture Devices` ***only*** *for now. You expand the assignment to* `PROD – ISE Posture Devices` *in the Expand phase.*

Step 6 — Map compliance to ISE authorization (architectural — defer to Cisco docs)

In the **Cisco ISE** admin console, build the authorization logic that turns the compliance signal into an access decision. At the architectural level, standardize on this mapping:

- **Device registered = true AND compliant = true** → authorization result **PermitAccess** (full network / VPN).
- **Device registered = true AND compliant = false** → authorization result **denies** or assigns a **remediation / quarantine** profile (limited VLAN, redirect to remediation).
- **Device registered = false (unmanaged)** → authorization result **denies** or redirects the user to the enrollment portal.

Use ISE's MDM compliance dictionary attributes (device registered, device compliant, and related fields returned from Intune) as the **conditions** in these authorization rules, and scope the rules to the relevant policy set (VPN, wired 802.1X, wireless).

> ***Vendor boundary:*** *the exact ISE authorization-profile and policy-set construction — dictionaries, attribute names, and rule editor —* ***must be followed from Cisco's official ISE documentation for your release.*** *Build the rules there; this solution defines the mapping intent and provides the Intune-side signals they consume.*

Step 7 — Assign to the pilot and stage the gate

1. Confirm the baseline compliance policies (Step 5) are assigned to `PILOT – ISE Posture Devices` only.

1. In ISE, scope the new compliance-gating authorization rules so they apply to the pilot population first (for example a policy set or condition limited to pilot devices/users), per Cisco's documentation.

1. Announce the change window (R7) to pilot users and confirm the rollback is ready.

Step 8 — Validate (pass/fail gate)

Run the validation before expanding. In summary (full pass/fail steps are in the Testing guide):

1. **Compliant pilot device** connects → ISE reads registered + compliant → **access granted**. *(Pass gate 1.)*

1. **Deliberately non-compliant pilot device** (for example encryption turned off, so Intune marks it non-compliant) connects → ISE reads non-compliant → **access denied / quarantined**. *(Pass gate 2 — the critical one.)*

1. Use `ISEPOST-Validation.ps1` to confirm the device's compliance/enrollment state as ISE reads it, and `ISEPOST-Discovery.ps1` to confirm connector readiness.

Do not expand until **both** gates pass: compliant → granted, and non-compliant → denied.

Step 9 — Expand

1. Create `PROD – ISE Posture Devices` (or run `ISEPOST-Set-ScopeTargeting.ps1`) and add the broader device population in controlled waves.

1. Extend the compliance policy assignments (Step 5) and the ISE authorization rule scope (Step 6) from pilot to `PROD – ISE Posture Devices`.

1. Expand in waves, watching the `ISEPOST-Report.ps1` compliance-coverage report between waves.

Keep the rollback available throughout the rollout.

Testing

Validate the integration with explicit **pass/fail gates** before you expand beyond the pilot. The core principle of this solution: **compliant → access granted; non-compliant → access denied.** Both directions must be proven.

Run these tests against `PILOT – ISE Posture Devices` only. Record the result of each gate.

Gate 1 — Connector readiness

Goal: ISE can authenticate to Entra and read Intune device state.

Steps:

1. Run `ISEPOST-Discovery.ps1`. Confirm the app registration

`ISE-Intune-Compliance-Connector` exists, has a valid (unexpired) secret, and shows the Graph permissions `DeviceManagementManagedDevices.Read.All` and `DeviceManagementConfiguration.Read.All` **admin-consented**.

1. In the ISE console, run **Test Connection** on the External MDM (Intune) server (per Cisco's

documentation).

Pass: Discovery reports the app, valid secret, and consented permissions; ISE Test Connection succeeds. **Fail:** Missing/expired secret, un-consented permission, or ISE Test Connection error → see Troubleshooting.

Gate 2 — Compliance policies present and evaluating

Goal: The named baseline policies exist, are assigned to the pilot, and Intune is producing a compliance verdict per device.

Steps:

1. Run `ISEPOST-Discovery.ps1` and confirm the baseline policies

(`Compliance – Windows – Baseline`, `Compliance – iOS – Baseline`, `Compliance – Android – Baseline`, `Compliance – macOS – Baseline`) are present and assigned to `PILOT – ISE Posture Devices`.

1. Intune → **Devices** → **Compliance** and confirm pilot devices show a computed compliance

state (compliant / non-compliant / in grace), not "not evaluated."

Pass: All target-platform baseline policies exist, assigned to pilot, with devices evaluating. **Fail:** A policy missing/unassigned, or devices stuck "not evaluated" → see Troubleshooting.

Gate 3 — Compliant device is GRANTED access (the positive path)

Goal: A healthy, enrolled pilot device gets full network / VPN access.

Steps:

1. Pick a pilot device that is **enrolled** and **compliant**. Confirm with

`ISEPOST-Validation.ps1` on that device (it reports enrollment + compliance as ISE reads it).

1. Have the device connect through an ISE-mediated path (VPN, wired 802.1X, or wireless).
2. Confirm in ISE **Live Logs** that the authorization result is the **permit / full-access**

profile and the device reaches the network.

Pass: Enrolled + compliant device → **access granted**, permit profile in Live Logs. **Fail:** A compliant device is denied → see Troubleshooting ("compliant device denied").

Gate 4 — Non-compliant device is DENIED access (the critical path)

Goal: Prove the gate actually enforces. This is the gate that justifies the whole solution.

Steps:

1. Take a pilot device and deliberately make it non-compliant against its baseline policy — for example **turn off disk encryption** (BitLocker / FileVault) or let it fall below the OS floor.
1. Force an Intune sync and confirm Intune now marks it **non-compliant**. Confirm with `ISEPOST-Validation.ps1` that the device reads as non-compliant.
1. Have the device connect through the ISE-mediated path.
2. Confirm in ISE **Live Logs** that the authorization result is the **deny / quarantine / remediation **profile and the device does** not** get full access.

Pass: Non-compliant device → **access denied or quarantined**. **This gate must pass. Fail:** A non-compliant device still gets full access → **stop the rollout** and see Troubleshooting ("non-compliant device allowed").

Gate 5 — Unmanaged device is DENIED / redirected

Goal: A device that is not enrolled in Intune is treated as unmanaged.

Steps:

1. Attempt to connect with a device that is **not enrolled** in Intune.
2. Confirm in ISE Live Logs the authorization result is **deny** or **enrollment redirect**, not full access.

Pass: Unmanaged device → denied or redirected to enrollment. **Fail:** Unmanaged device granted full access → see Troubleshooting.

Gate 6 — Fleet coverage snapshot

Goal: Understand compliance coverage and the conceptual access decision across the pilot before expanding.

Steps:

1. Run `ISEPOST-Report.ps1` and review the compliance-coverage + conceptual access-decision report (how many devices would be granted vs denied).

Pass: Coverage is understood; the granted/denied split matches expectations. **Fail:** Unexpected large denial population → investigate compliance policy scope / grace before expanding.

Exit criteria to expand

Proceed to **Expand** only when **Gates 1–5 all pass** — in particular **Gate 3 (compliant → granted)** and **Gate 4 (non-compliant → denied)** — and the Gate 6 coverage report looks sane. Keep the rollback (R7) available through expansion.

Troubleshooting

Work top to bottom — the first sections cover the failures that block everything else (authentication and connector), the later sections cover the two dangerous logic failures (compliant device denied, non-compliant device allowed). For the Intune/Entra/Graph side use the diagnosis steps below; for anything inside the **Cisco ISE console**, follow **Cisco's official documentation** — those internals are deferred to the vendor.

1. App registration / permission errors

Symptom: ISE's Test Connection fails with an authentication or authorization error; or `ISEPOST-Discovery.ps1` reports the app, secret, or permissions as FAIL.

Diagnose:

- Run `ISEPOST-Discovery.ps1`. It checks the app `ISE-Intune-Compliance-Connector`, secret expiry, the Graph permissions, and admin consent.

Common causes and fixes:

- **Wrong client/tenant id or secret in ISE** — re-copy the Application (client) ID and Directory (tenant) ID from the app's Overview, and re-generate the client secret if unsure; paste the secret **Value** (not the Secret ID) into ISE. (Implementation Steps 1, 3.)
- **Expired client secret** — Discovery flags this as FAIL/WARN. Create a new secret and update it in ISE. An expired secret is the most common "it worked last quarter" outage.
- **Permission missing or not consented** — the Graph permissions `DeviceManagementManagedDevices.Read.All` and `DeviceManagementConfiguration.Read.All` must be present **and** show "Granted for <tenant>". Add them and click **Grant admin consent** (Implementation Step 2).
- **Delegated instead of Application permissions** — ISE runs unattended; the permissions must be **Application** type, not Delegated. Remove delegated, add application, re-consent.

2. Connector not reporting (ISE reads no compliance state)

Symptom: ISE authenticates but gets no device data, or every device looks unmanaged; Test Connection may pass yet authorization sees empty MDM attributes.

Diagnose:

- Confirm the device is actually enrolled: run `ISEPOST-Validation.ps1` on the device (or check Intune → Devices → All devices).

• Confirm reachability: the connector is an outbound HTTPS call from ISE to Microsoft Graph / Entra (Requirement R5).

Common causes and fixes:

• **Egress / TLS interception** — a firewall blocking 443 to Microsoft, or a proxy that breaks the TLS chain, stops the connector. Open outbound 443 to the Graph/Entra endpoints and ensure no TLS interception on that path.

• **Missing Microsoft CA in ISE trust store** — import the required Microsoft root/intermediate CAs into ISE's trusted certificate store (per Cisco's documentation), then retest.

• **Device not enrolled** — an unenrolled device has no state to report; enroll it (R1).

• **Read permission missing** — without `DeviceManagementManagedDevices.Read.All` the connector authenticates but reads nothing. Re-check Section 1.

3. Compliant device denied (false lockout — positive path broken)

Symptom: A device that Intune shows as **compliant** is still denied network / VPN access by ISE. This generates user complaints and must be fixed fast.

Diagnose:

• Run `ISEPOST-Validation.ps1` on the device. If it reports **enrolled + compliant** but ISE still denies, the break is between the compliance verdict and the ISE decision.

• Check the device's **last Intune sync** time (Validation prints it). A stale device may have an old non-compliant verdict cached.

Common causes and fixes:

• **Stale compliance state** — the device fixed the issue but hasn't re-synced. Force a sync (`ISEPOST-Remediation.ps1`, or Company Portal → sync) and retry.

• **Grace vs compliant mismatch** — if you intended in-grace devices to be granted, confirm the ISE rule treats `inGracePeriod` as compliant; if it only permits strict `compliant`, in-grace devices are denied. Align the rule or the grace policy (Best Practices).

• **ISE authorization rule mismatch** — the rule's registered/compliant condition or rule order may be wrong so a compliant device falls through to a deny. Review the ISE authorization policy per **Cisco's documentation** (rule order, condition attributes, the permit profile).

• **Policy not assigned to the device** — if no compliance policy targets the device, Intune may report "not evaluated" rather than compliant. Confirm the baseline policy is assigned to the device's group (`ISEPOST-Discovery.ps1` shows assignment counts).

4. Non-compliant device allowed (enforcement gap — critical)

Symptom: A device that Intune shows as **non-compliant** still gets full network / VPN access. **This defeats the entire solution — stop the rollout and fix it before expanding.**

Diagnose:

• Run `ISEPOST-Validation.ps1` on the offending device — confirm Intune actually marks it non-compliant (if Intune says compliant, the problem is your compliance policy, not ISE).

• Confirm the connecting path is actually ISE-mediated (that the VPN/switch/WLC sends auth to ISE and honors the returned authorization result).

Common causes and fixes:

• **Compliance policy too weak / not assigned** — if the device is non-compliant only in intent but Intune reports it compliant, the baseline policy is missing the rule or isn't assigned to the device. Tighten and assign the `Compliance – <platform> – Baseline` policy (Implementation Step 5); `ISEPOST-Discovery.ps1` confirms presence and assignment.

• **Grace period masking it** — an `inGracePeriod` device is intentionally still granted. If you need immediate denial for a high-risk signal (encryption off, jailbreak/root), remove or shorten grace on that rule (Best Practices).

• **ISE rule not evaluating MDM compliance** — the authorization rule may permit before checking the compliant attribute, or a broader permit rule sits above it. Fix rule order/conditions in the ISE authorization policy per **Cisco's documentation** so the non-compliant path reaches a deny/quarantine result.

• **Network device bypassing ISE** — a VPN headend or switchport not enforcing the ISE result (or a fallback/open-auth mode) lets devices on regardless. Confirm enforcement on the network access device per its vendor documentation.

5. Quick reference — which tool for which symptom

Symptom	First tool
Auth / permission / secret error	`ISEPOST-Discovery.ps1`
One device's state wrong	`ISEPOST-Validation.ps1`
Fleet-wide granted/denied split off	`ISEPOST-Report.ps1`
Non-compliant devices need nudging	`ISEPOST-Remediation.ps1` (`-WhatIf` first)
Pilot/prod scope group missing	`ISEPOST-Set-ScopeTargeting.ps1`
Anything inside the ISE console	Cisco's official ISE documentation

Best Practices

Operate the posture gate so it enforces security without generating false lockouts. These practices cover grace periods, remediation, secret hygiene, and steady-state operation.

Grace periods — enforce, but don't ambush

• Set a **grace period** on each compliance policy's *Actions for noncompliance* rather than marking devices non-compliant instantly. A user whose patch level slips at 9am should get a remediation window, not an immediate VPN lockout. A grace of **24–72 hours** is a common starting point; tune per platform and risk.

• During grace, Intune reports the device as **in grace period** (still treated as compliant by the gate). ISE therefore still grants access, but the user is being warned.

• Pair grace with a **"Send email to end user"** noncompliance action so the user knows what to fix and by when.

• Shorten or remove grace for the highest-risk signals (jailbreak/root, encryption off) — those warrant faster denial than a lagging OS patch.

Remediation — give users a path back

• Make the **non-compliant / quarantine** ISE authorization result a **remediation VLAN** with reachability to the enrollment portal, patch/update services, and support — not a dead end. A user who is denied should be able to self-heal.

• Publish clear end-user guidance (Help Center) for each common failure: how to turn encryption back on, update the OS, or re-enroll. Denial without a fix path generates support tickets.

• Use `ISEPOST-Remediation.ps1` (default `-WhatIf`) to identify non-compliant devices and nudge them toward compliance (for example triggering a sync / notification) before enforcement bites.

Naming and reuse

• Keep every object on the fixed convention — `ISE-Intune-Compliance-Connector`, `Compliance – <platform> – Baseline`, `PILOT – ISE Posture Devices`, `PROD – ISE Posture Devices` — so the connector, policies, and scripts stay coherent and auditable. Do not create ad-hoc duplicates.

Secret hygiene

• The app registration's **client secret** is a live credential. Store it in a secrets manager, never in a repo or ticket. Record its **expiry** and renew **before** it lapses — an expired secret silently breaks the connector and every device starts reading as unmanaged.

• Prefer the **shortest practical secret lifetime** your operations can renew reliably. Set a calendar reminder ahead of expiry.

• Grant only the **read** Graph permissions the connector needs; do not over-scope the app.

Pilot-first, always

• Never widen the compliance policy assignment or the ISE authorization scope straight to the fleet. Expand `PILOT` → `PROD` in **waves**, validating each wave with `ISEPOST-Report.ps1`.

• Keep the **rollback** (loosen the ISE rule to permit / unassign the policy) documented and reachable throughout every wave.

Steady-state operations

• Run `ISEPOST-Discovery.ps1` on a schedule to catch a **secret nearing expiry** or a permission that lost consent before it causes an outage.

• Run `ISEPOST-Report.ps1` regularly to watch compliance coverage and the granted/denied split trend; a sudden spike in denials usually means a policy change or an expired secret, not a fleet-wide device failure.

• Review compliance policy rules periodically against your security baseline (OS floors move, new signals appear) and re-validate the pass/fail gates after any change.

• Keep the ISE side aligned with **Cisco's current documentation** across ISE upgrades — the MDM integration and its dictionary attributes can change between releases.

Deliverables Summary

This solution delivers everything needed to wire Cisco ISE to Microsoft Intune through the MDM compliance connection, so ISE gates network / VPN access on real-time device compliance and enrollment — deployed pilot-first and rollback-ready.

Documentation

Document	What it gives you
Solution Overview	How ISE-to-Intune posture works: the Entra app registration ISE uses, the MDM compliance connection, the compliance/enrollment signals ISE reads, and how they map to authorization.
Requirements Guide	Every prerequisite (Intune + enrolled devices, compliance policies, the Entra app registration with Graph permissions, ISE licensing/version, network reachability, pilot group, change control) with Description / Why / Verify / Implement-if-missing.
Implementation Guide	Ordered, click-by-click steps: register the app in Entra, grant Graph permissions, create the secret, connect ISE to Intune (architectural), author the named compliance policies, map compliance to ISE authorization (architectural), assign the pilot, validate, and expand.

Document	What it gives you
Testing Guide	Explicit pass/fail gates: connector readiness, policies evaluating, compliant → granted, non-compliant → denied, unmanaged → denied, fleet coverage.
Troubleshooting Guide	Fixes for app-registration/permission errors, connector not reporting, compliant device denied, and non-compliant device allowed.
Best Practices	Grace periods, remediation paths, secret hygiene, pilot-first expansion, steady-state operations.
Global Requirements (Network)	Category-level prerequisites shared by all Network solutions.

PowerShell scripts

All scripts are read-only against Microsoft Graph (the remediation script defaults to `-WhatIf`). Cisco ISE internals are deferred to Cisco's documentation.

Script	Purpose
ISEPOST-Discovery.ps1	Connector readiness — the Entra app registration for ISE, its Graph permissions and secret state, and the Intune compliance policies.
ISEPOST-Validation.ps1	Confirms THIS device's compliance + enrollment state exactly as ISE would read it from Intune.
ISEPOST-Report.ps1	Fleet compliance-coverage report plus a conceptual access-decision (granted vs denied) breakdown.
ISEPOST-Remediation.ps1	Safely nudges non-compliant devices toward compliance (default `-WhatIf`; nothing changes until you pass `-Execute`).
ISEPOST-Set-ScopeTargeting.ps1	Creates and populates the pilot / expansion Entra group(s) used for staged rollout.

Named objects this solution standardizes

- **Entra app registration:** `ISE-Intune-Compliance-Connector`
- **Compliance policies:** `Compliance – Windows – Baseline`, `Compliance – iOS – Baseline`,

`Compliance – Android – Baseline`, `Compliance – macOS – Baseline`

- **Pilot group:** `PILOT – ISE Posture Devices`
- **Expansion group:** `PROD – ISE Posture Devices`

Outcome

Network and VPN access is granted only to devices Intune says are compliant and enrolled — enforced by Cisco ISE, and proven on a pilot (compliant → granted, non-compliant → denied) before you scale to the fleet.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

ISEPOST-Discovery.ps1

Purpose: Read-only readiness check for the ISE-to-Intune posture connector. Confirms the Entra app registration named 'ISE-Intune-Compliance-Connector' exists, reports its Microsoft Graph application permissions and client-secret expiry, and lists the Intune device compliance policies ISE's decision depends on. Nothing is changed. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\ISEPOST-Discovery.ps1

How to run: powershell -ExecutionPolicy Bypass -File .\ISEPOST-Discovery.ps1

ISEPOST-Remediation.ps1

Purpose: Finds Intune managed devices that are non-compliant (the population ISE would deny) and nudges them toward compliance by triggering an Intune sync so they re-evaluate and re-report. Runs in preview by default: it lists what it WOULD do and changes nothing. Pass -Execute to actually send the sync action. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\ISEPOST-Remediation.ps1 powershell -ExecutionPolicy Bypass -File .\ISEPOST-Remediation.ps1 -Execute

How to run: powershell -ExecutionPolicy Bypass -File .\ISEPOST-Remediation.ps1 powershell -ExecutionPolicy Bypass -File .\ISEPOST-Remediation.ps1 -Execute

ISEPOST-Report.ps1

Purpose: Read-only fleet report. Enumerates Intune managed devices via Microsoft Graph and summarizes enrollment + compliance coverage, then applies this solution's mapping (compliant+enrolled = granted; else denied/quarantine) to show a conceptual access-decision split ISE would enforce. Optionally writes a CSV. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\ISEPOST-Report.ps1 powershell -ExecutionPolicy Bypass -File .\ISEPOST-Report.ps1 -CsvPath .\ise-posture-report.csv

How to run: powershell -ExecutionPolicy Bypass -File .\ISEPOST-Report.ps1 powershell -ExecutionPolicy Bypass -File .\ISEPOST-Report.ps1 -CsvPath .\ise-posture-report.csv

ISEPOST-Set-ScopeTargeting.ps1

Purpose: Creates the Entra security group 'PILOT – ISE Posture Devices' (and optionally 'PROD – ISE Posture Devices') used to scope the compliance policies and ISE authorization rules pilot-first. Can add members you supply by object id. Runs in preview by default: it reports what it WOULD create/add and changes nothing until you pass -Execute. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\ISEPOST-Set-ScopeTargeting.ps1 powershell -ExecutionPolicy Bypass -File .\ISEPOST-Set-ScopeTargeting.ps1 -Execute -MemberObjectIds id-1,id-2 powershell -ExecutionPolicy Bypass -File .\ISEPOST-Set-ScopeTargeting.ps1 -Execute -IncludeProd

How to run: powershell -ExecutionPolicy Bypass -File .\ISEPOST-Set-ScopeTargeting.ps1 powershell -ExecutionPolicy Bypass -File .\ISEPOST-Set-ScopeTargeting.ps1 -Execute -MemberObjectIds id-1,id-2 powershell -ExecutionPolicy Bypass -File .\ISEPOST-Set-ScopeTargeting.ps1 -Execute -IncludeProd

ISEPOST-Validation.ps1

Purpose: Read-only per-device check. Looks up the machine it runs on (by Entra device id, or a device name you pass) in Intune via Microsoft Graph and reports the enrollment and compliance state ISE would consume for its access decision, plus a conceptual grant/deny verdict. Confirms the positive and negative test gates from the device side. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\ISEPOST-Validation.ps1 powershell -ExecutionPolicy Bypass -File .\ISEPOST-Validation.ps1 -DeviceName your-device-name

How to run: powershell -ExecutionPolicy Bypass -File .\ISEPOST-Validation.ps1 powershell -ExecutionPolicy Bypass -File .\ISEPOST-Validation.ps1 -DeviceName your-device-name